

LAB-03 Component Diagram

=====

Name: Rahul Saran Sharma

SRN: PES1UG24AM218

Class: 'AIML-D'

Date: 09/09/2026

=====

Personalized Meal & Diet Subscription Manager

Architecture Selection: We chose Client-Server Architecture for the Personalized Meal & Diet Subscription Manager.

Why not Layered: Layered architecture organizes a single application into internal tiers (presentation/business/data) — it doesn't naturally model two distinct client apps (Subscriber, Dietitian) talking to a shared backend over the network, which is exactly the access pattern this system needs.

Why not Microservices: Splitting Meal Plan Generation, Health Data, and Delivery into independently deployed microservices adds network latency and operational complexity that this subscription-scale system doesn't need, and makes it harder to guarantee one consistent, auditable meal-plan state (Draft → Approved) across services — a Client-Server model gives that consistency for free.

Reason 1 – Centralized control over clinical health data

The system manages sensitive metabolic profiles, allergen exclusions, and vitals for every subscriber, and dietitians must review this data with role-based access. A Client-Server model keeps all subscriber health records and business logic on a central server, so RBAC, AES-256 encryption at rest, and audit logging (dietitian ID + timestamp on every approval) can be enforced in one place, rather than duplicated or drifting across independently deployed services.

Reason 2 – Single source of truth for the clinical approval workflow

Dietitians must see one authoritative, consistent state for a subscriber's meal plan (Draft -> Reviewed -> Approved) before it moves to kitchen dispatch. A central server guarantees this consistency for both the Subscriber Client App and the Dietitian Dashboard Client, avoiding the state-synchronization problems that a more distributed, independently-deployed style would introduce for a workflow that requires a strict, auditable hand-off between roles.

Security Advantage

Because the Subscriber Client App and Dietitian Dashboard Client hold no persistent copies of health records, a compromised or lost client device does not expose the underlying database. All requests to the Subscriber & Health Data Server pass through TLS 1.3 in transit and are checked against RBAC policy before any clinical data is returned, concentrating the system's security boundary at the server rather than across many client endpoints.

Performance Benefit

Centralizing meal-plan generation on the Meal Plan Generation Server lets it maintain shared, precomputed nutrient and allergen-validation data in server memory rather than re-fetching it per client request. This server-side optimization (connection pooling, caching subscriber constraints) is what allows the system to meet its 2.5-second, 95th-percentile generation target under 500 concurrent requests, since the expensive computation is done once centrally instead of being duplicated across distributed nodes.

Requirements Traceability – FR/NFR to Architecture Mapping

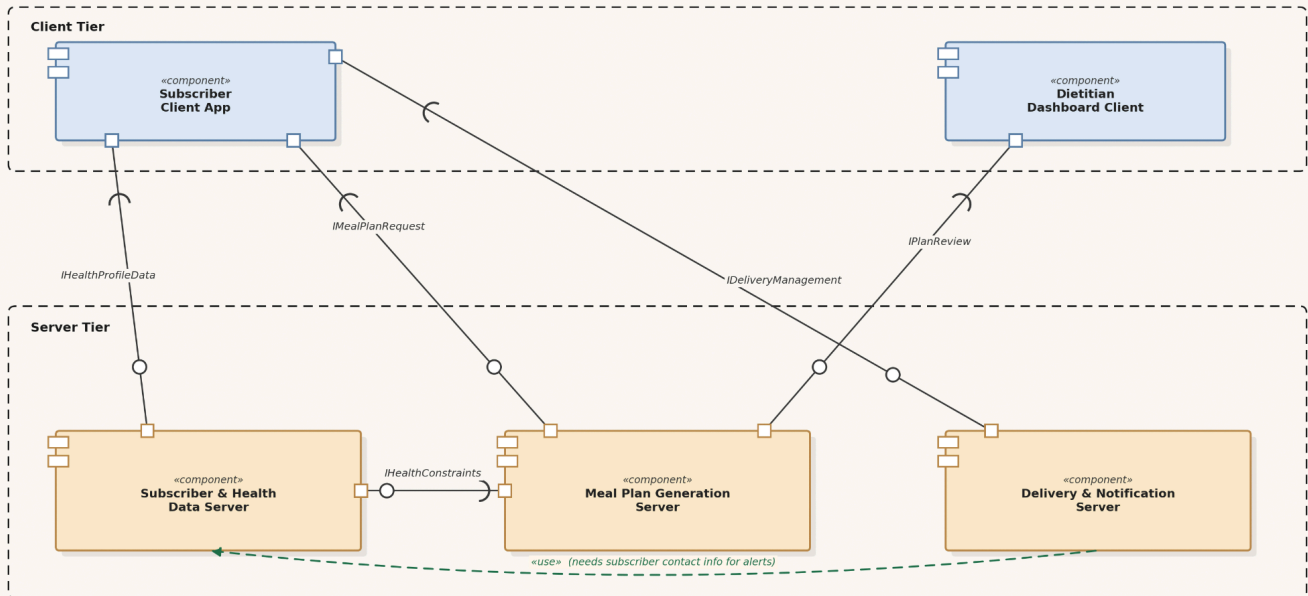
ID	Type	Description	Priority	How the Architecture Addresses It
FR-001	Functional	Generate weekly meal plan within calorie goal & allergen exclusions.	High	Meal Plan Generation Server executes this over IMealPlanRequest (from Subscriber Client App) and IHealthConstraints (from Subscriber & Health Data Server).
FR-002	Functional	Dietitian reviews, modifies, and approves a meal plan before dispatch.	High	Dietitian Dashboard Client invokes IPlanReview on the Meal Plan Generation Server; server centrally records approval state, dietitian ID, and timestamp.
FR-003	Functional	Subscriber logs daily glucose, weight, and blood pressure.	Medium	Subscriber Client App writes to the Subscriber & Health Data Server via IHealthProfileData, keeping one authoritative health record.
FR-004	Functional	Subscriber pauses, skips, or reschedules a delivery > 12h before dispatch.	High	Subscriber Client App calls IDeliveryManagement on the Delivery & Notification Server, which enforces the 12-hour cutoff centrally.
FR-005	Functional	Track daily delivery status and notify the subscriber.	Medium	Delivery & Notification Server owns status tracking and uses the «use» dependency on the Health Data Server to fetch subscriber contact details for alerts.
NFR-001	Performance	Generate a weekly plan within 2.5s at 500 concurrent requests.	High	Centralizing generation on one Meal Plan Generation Server allows shared

				caching/connection pooling – the optimization this NFR needs is done once, not duplicated per client.
NFR-002	Security & Compliance	Protect health data with AES-256 at rest, TLS 1.3 in transit, and RBAC.	High	Enforced entirely on the Subscriber & Health Data Server; clients never hold persistent copies of health records, so the security boundary is server-side.

Component Diagram

Personalized Meal & Diet Subscription Manager

UML Component Diagram — Client-Server Architecture



5 components • 5 Interfaces (IHealthProfileData, IMealPlanRequest, IPlanReview, IHealthConstraints, IDeliveryManagement) — exceeds the min. 4 required